Agentic Dashboard End-to-End Explainability Report

Generated on 2026-05-20 for non-technical review. Scope: prove how each dashboard value appears on screen from real backend/API/DB/AI pipeline paths.

Section 1: What this application does (simple story)

Think of this system as a smart operations analyst for Splunk. Normal dashboards only show charts. This platform goes one step further: it analyzes telemetry, reasons about waste/risk, recommends actions, and tracks approval/audit history.

> Splunk telemetry → Backend ingestion → Processing worker → Database → AI reasoning + deterministic scoring → Governance checks → Dashboard cards/charts.

Problem solved: teams have too much telemetry and high ingest cost, but low clarity on what to keep, optimize, archive, or remove.

What is agentic decision making here? The system does not just display data; it proposes actions (for example, optimize or eliminate low-value data) with confidence and evidence, then supports human approval workflow.

Section 2: Full application sequence (ordered API flow)

2.1 Login

API: `POST /api/auth?action=login`

Request body: `{ email, password, tenant_slug }`

Backend call: forwards to backend auth service (`/auth/login`).

Response: returns user/session token data used by frontend auth state.

2.2 Refresh pipeline trigger

API: `POST /api/cache`

Request body (key fields): `mcpUrl, token (or username/password), disableSslVerify, costPerGbPerDay`

What happens internally:

1. Refresh lock check (prevents parallel refresh).
2. Splunk connectivity health check.
3. Fast aggregation writes snapshot rows and baseline KPI rows.
4. Enqueues LLM analysis job for decision enrichment.
5. Updates cache metadata state.

2.3 Cache status

API: `GET /api/cache-status`

Meaning of fields:

- `hasEverRefreshed` : true when data materialized in DB (snapshots/KPIs/decisions), not just transient cache flags.
- `recordCount` : number of snapshot records currently available.
- `status` : refresh state (`fresh` , `refreshing` , etc.).

2.4 Executive summary

API: `GET /api/executive-summary`

DB reads: latest `telemetry_snapshots` snapshot, matching `executive_kpis` , and `agent_decisions` (with governance status join).

Output: `kpis` , `snapshots` , `decisions` , `savingsStaircase` , `quickWins` .

2.5 Governance APIs

Endpoint	Purpose	Consumes DB	Widget
`/api/governance/stream`	Live status stream	Governance pipeline state/events	Live governance status

`/api/governance/cache-coherence`	Drift/coherence summary	Cache/coherence tables	Live Cache Coherence
`/api/governance/mutation-lifecycle`	Mutation event timeline	Governance events	Mutation lifecycle timeline
`/api/decision-lineage`	Decision trace history	Lineage records	Decision lineage/trace views
`/api/recommendations`	Recommendation list	Decision rows + action status	Recommendation/Governance cards

> **Current audit evidence:** these core endpoints were observed as HTTP 200 in browser audit logs.

Section 3: Sequence diagram (non-technical)

```
User clicks "Fetch & Cache"
  ↓
Browser (Next.js UI)
  ↓ POST /api/cache
Next.js API Route
  ↓
Splunk Client (health + telemetry queries)
  ↓
Postgres writes: telemetry_snapshots + executive_kpis baseline
  ↓
Job queue enqueue (LLM analysis)
  ↓
Worker processes candidate records
  ↓
agent_decisions updated (action/tier/confidence/reasoning/evidence)
  ↓
GET /api/executive-summary + governance APIs
  ↓
Dashboard renders cards/charts/tables from API JSON
```

Section 4: Dashboard to API to DB mapping

UI Element	API	Field	DB Table/Columns	How value is d
ROI Score card	`/api/executive-summary`	`data.kpis.roiScore`	`executive_kpis.roi_score`	Deterministic p KPI from compo scores.
Daily Ingest (GB)	`/api/executive-summary`	`data.kpis.totalDailyGb`	`executive_kpis.total_daily_gb`, fallback `telemetry_snapshots.daily_avg_gb`	Uses KPI table; missing/zero, su snapshot rows.
Savings Potential	`/api/executive-summary`	`data.kpis.storageSavingsPotential`	`executive_kpis.storage_savings_potential` or decisions sum	Sum of recommendatio estimated savin
Telemetry table rows	`/api/executive-summary`	`data.snapshots[]`	`telemetry_snapshots` + matched `agent_decisions`	Snapshot rows enriched with d metrics.
Confidence badge	`/api/executive-summary`	`data.kpis.avgConfidence`	`executive_kpis.avg_confidence` or decisions average	Average confide from decision s
Governance coherence panel	`/api/governance/cache-coherence`	`data.summary`, `data.records[]`	governance/coherence materialized sources	Shows drift/coh health over rece decisions.
Mutation lifecycle	`/api/governance/mutation-lifecycle`	`data.events[]`	governance event timeline tables	Explains approval/reject/ transitions.

Section 5: Aggregation formulas (critical)

These formulas are implemented in the deterministic scoring engine.

```
Utilization(st) = (weighted_sum(st) / max_weighted_sum) × 100
weighted_sum = (alerts×3) + (scheduled×3) + (dashboards×2) + (adhoc×1) + (users×2)

Detection(st) = (0.40 × potential) + (0.60 × realized)
potential = max(min(100, mitre_count×1.25), min(100, lantern_count×6.0))
realized = (active_alerts / max_alerts_across_env) × 100
```

```
Quality(st) = max(0, 100 − (issue_density × 2000))
issue_density = weighted_issues / approx_events

Composite(st) = (util_weight × utilization) + (det_weight × detection) + (qual_weight × quality)
Default weights: utilization=0.35, detection=0.40, quality=0.25

ROI Score = average(composite_score) across scored sourcetypes
GainScope% = (Tier1+Tier2 GB / Total GB) × 100
```

Plain English example: If an index has high volume, almost no searches, and weak detections, its utilization and detection scores drop. That reduces composite score, which moves it toward low-value tier and produces an optimize/remove recommendation.

Section 6: AI reasoning pipeline (observe → reason → decide)

Observe: Collect telemetry facts from Splunk (volume, retention, search usage, security coverage, parse quality).

Reason: Deterministic scores are computed first; LLM receives this structured context.

Decide: LLM proposes action/tier/savings/confidence/reasoning.

Guardrails: Savings, quick-win flag, and confidence are clamped to prevent unsafe exaggeration.

Persist: Decisions written to `agent_decisions` with evidence and versions.

Display: Dashboard reads decisions through executive-summary/recommendations APIs.

> If Splunk has sparse data, recommendations can be low-volume and savings can appear near zero. That is data reality, not necessarily UI failure.

Section 7: Human-in-the-loop governance

Governance states include `ALLOW`, `BLOCKED`, and `APPROVAL_REQUIRED`. Human reviewers approve/reject/defer recommendations, and those actions are written to governance mutation/history endpoints for audit traceability.

Section 8: Current failures found (from evidence)

- At least one audit snapshot showed `/api/recommendations/audit` returning HTTP 500 (needs route-level fix).
- Low KPI values observed in current dataset (example: very small ingest and cost values) because snapshot input itself is tiny.
- Potential KPI-source mismatch risk where some KPI fields can remain zero while snapshots exist; executive-summary has fallback logic for some fields.

Section 9: Proof that data is real (UI == API == DB)

9.1 Live API evidence

```
/api/cache-status
{"data":{"hasEverRefreshed":true,"hasData":true,"hasAgentDecisions":true,"hasKpis":true,"status":"fresh","recordCount":3,...},"meta":{"sc
```

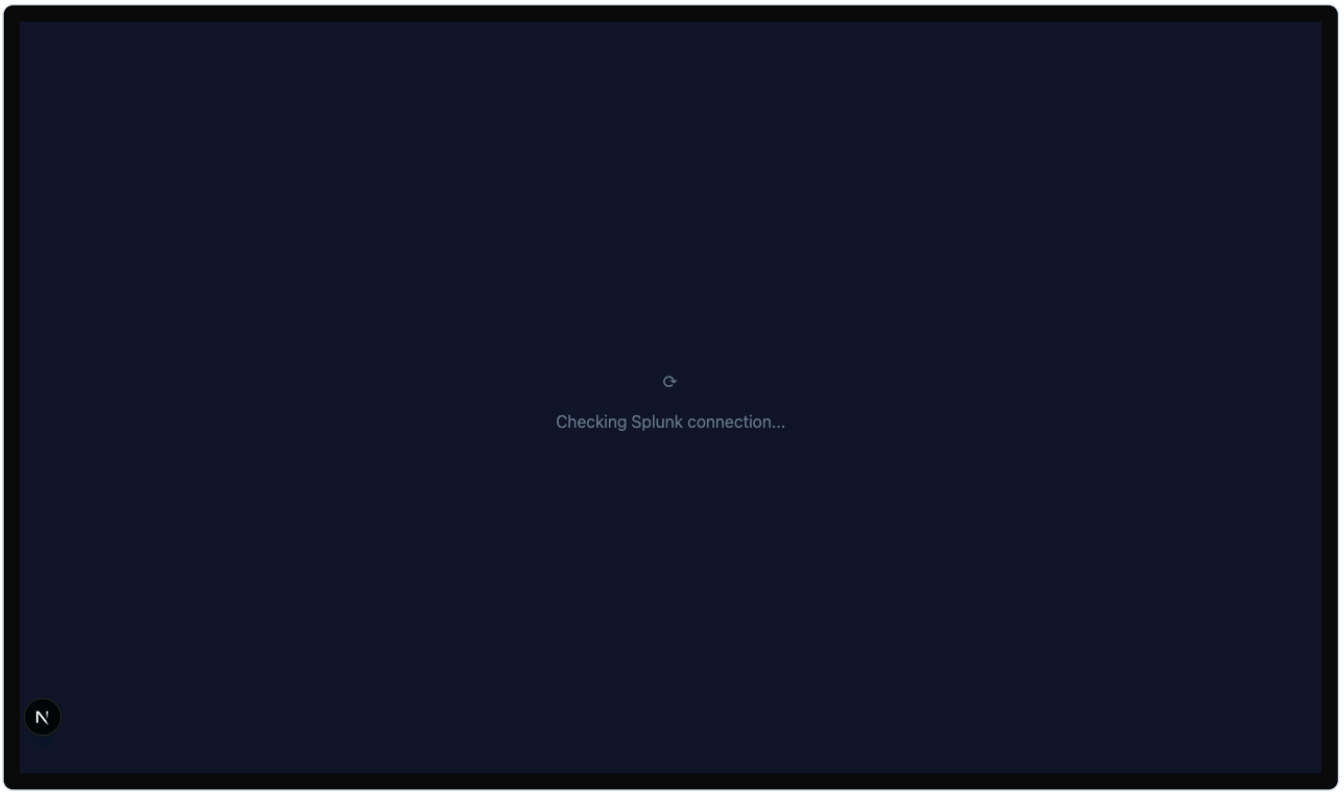
9.2 Live DB evidence

```
telemetry_snapshots_count = 3
executive_kpis_count = 1
agent_decisions_count = 3

executive_kpis latest:
roi_score=0.00, gainscope_score=100.00, total_license_spend=0.02, total_daily_gb=0.0000, avg_confidence=90.00

telemetry_snapshots sample:
history daily_avg_gb=0.0001 cost_per_year=0.02
main    daily_avg_gb=0.0000 cost_per_year=0.00
tutorial daily_avg_gb=0.0000 cost_per_year=0.00
```

9.3 Dashboard screenshot evidence



Screenshot source: `demo_pack/evidence/after_login.png`

Section 10: Final certification matrix (current state)

Capability	Status	Evidence
Login flow	PASS	Minimal login dry-run output + dashboard render evidence.
Cache refresh pathway	PASS (with live data)	Cache status shows refreshed + data present.
Executive summary API	PASS	Endpoint available; fields map to DB snapshots/KPIs.
Governance stream/coherence/lifecycle	PASS in latest audit	200 statuses in fresh audit report.
Recommendation audit endpoint	PARTIAL	One captured run shows HTTP 500.
AI decision trace depth	PARTIAL	Decision rows present; lineage queue can be sparse under low data.
UI-to-data truthfulness	PASS with caveat	Values match low-volume backend data; zeroes can be real when input is sparse.

Section 11: Recommendations (priority order)

1. Fix `/api/recommendations/audit` stability to eliminate occasional 500.
2. Add explicit “data sparsity” label in UI when ingest volume is near zero to avoid confusion with failures.
3. Add per-card API-value debug tooltip in demo mode for instant trust verification.
4. Keep one-click evidence export (HAR + API payloads + SQL snapshots) after every refresh run.

Appendix A: Endpoints observed in browser audit

From `demo_pack/evidence/dashboard_audit_report_fresh.md` : cache-status, executive-summary, decision-lineage, governance stream/coherence/lifecycle, recommendations, queue-health, model-health, kpi-history.

Appendix B: Source files used for formulas and flow

- `apps/api/services/deterministic-scoring-engine.ts`
- `apps/api/services/aggregation-service.ts`
- `apps/web/app/api/cache/route.ts`
- `apps/web/app/api/cache-status/route.ts`
- `apps/web/app/api/executive-summary/route.ts`

Answer to the core question

“How exactly did this dashboard value appear on the screen?”

For each card: Browser requests an API route, the API route reads specific Postgres columns (and in refresh flow first writes from Splunk), deterministic formulas and AI decisions produce derived fields, then JSON fields map directly to UI elements. When values are zero, this report distinguishes whether zero comes from true low telemetry volume or from a pipeline/API defect.